VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

MÃ QUỐC CƯỜNG – 522I0001
NGUYỄN CAO PHI – 521V0006

Object-Oriented Solutions to the Problem of Mining Top-K Maximal Frequent Itemsets from Uncertain Databases

Information Technology Project – 504091

HO CHI MINH CITY, YEAR 2026

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

MÃ QUỐC CƯỜNG – 522I0001
NGUYỄN CAO PHI – 521V0006

Object-Oriented Solutions to the Problem of Mining Top-K Maximal Frequent Itemsets from Uncertain Databases

Information Technology Project – 504091

Advised by
Prof. Nguyễn Chí Thiện

HO CHI MINH CITY, YEAR 2026

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Professor Nguyễn Chí Thiện for his patient guidance, thoughtful supervision, and continuous encouragement throughout this project. His expertise in data mining and algorithm design provided invaluable direction at every stage of our work.

We also thank our classmates for their helpful discussions and feedback, and Ton Duc Thang University for providing the academic environment and resources that made this project possible.

Ho Chi Minh City, day month year 2026

Authors

(Signature and full name)

DECLARATION OF AUTHORSHIP

I hereby declare that this thesis was carried out by my team under the guidance and supervision of Prof Nguyễn Chí Thiện; and that the work and the results contained in it are original and have not been submitted anywhere for any previous purposes. The data and figures presented in this thesis are for analysis, comments, and evaluations from various resources by my own work and have been duly acknowledged in the reference part.

In addition, other comments, reviews and data used by other authors, and organizations have been acknowledged, and explicitly cited.

I will take full responsibility for any fraud detected in my thesis. Ton Duc Thang University is unrelated to any copyright infringement caused on my work (if any).

Ho Chi Minh City, day month year
Author

[Signature and full name]

ABSTRACT

Frequent itemset mining is a fundamental task in data mining. Classical algorithms assume binary transaction data where items are either present or absent. However, real-world databases often contain **uncertainty**: a barcode scanner might be only 80% confident that it read a product correctly, a sensor might report a reading with known error margins, or data merged from multiple sources might contain conflicting entries. In such databases, each item is associated with an **existential probability** rather than a simple yes/no flag.

This report addresses the problem of mining **Top-K maximal frequent itemsets** from uncertain transaction databases using object-oriented design and two complementary support models. We implement and evaluate four algorithms:

Under the **expected support model**: **UGenMax** adapts GenMax (Gouda and Zaki, 2005) using bottom-up weighted tidset intersections, and **UFPMax** adapts FPMax (Grahne and Zhu, 2003) using top-down conditional database projection. Both employ branch-and-bound search with anti-monotonicity pruning, progressive focusing, and dynamic Top-K threshold raising.

Under the **probabilistic support model**: **TODIS-MAX** (based on Sun et al., 2010) provides exact probabilistic frequency computation via Dynamic Programming, and **APFI-MAX** (based on Chen et al., 2020) provides faster approximate estimation via the Central Limit Theorem.

All algorithms are implemented in pure Java SE with no external dependencies, following object-oriented design principles with separate algorithm classes (no main method) and experiment classes (with main method and statistics output). Experiments on three benchmark datasets (mushroom, retail, accidents) at medium uncertainty show that: (1) UGenMax and UFPMax produce identical results, with UGenMax being 19–43% faster on the primary retail dataset, 42–66% faster on mushroom, and 13–36% faster on accidents at moderate/high thresholds; (2) TODIS-MAX and APFI-MAX produce matching results, with APFI-MAX being 12–17$\times$ faster on mushroom and using significantly less memory on all datasets; (3) all algorithms scale to 340K transactions.

Contents

ACKNOWLEDGMENT **1**

DECLARATION OF AUTHORSHIP **2**

ABSTRACT **3**

TABLE OF CONTENTS **6**

IMAGE TABLE **7**

List of Abbreviations **8**

1 INTRODUCTION **9**

1.1 What is Frequent Itemset Mining? 9
1.2 The Challenge of Uncertainty 9
1.3 Why Maximal Itemsets? 9
1.4 Project Objectives 10
1.5 Report Structure 10

2 LITERATURE REVIEW **11**

2.1 Frequent Itemset Mining in Deterministic Databases 11
2.2 Mining Maximal Frequent Itemsets 11
2.3 Uncertain Database Mining 11
2.4 Probabilistic Support Mining 11
2.5 Gap and Our Contribution 12

3 PROBLEM STATEMENT AND RELATED DEFINITIONS **13**

3.1 Uncertain Transaction Database 13
3.2 Expected Support 13
3.3 Anti-Monotonicity Property 14
3.4 Frequent and Maximal Frequent Itemsets 14
3.5 Probabilistic Support 14
3.6 Problem Statement 15

4 METHODS **16**
4.1 Branch-and-Bound Framework 16
4.1.1 Pruning 1: Anti-Monotonicity Bound 16
4.1.2 Pruning 2: Progressive Focusing 16
4.1.3 Pruning 3: Top-K Dynamic Threshold Raising 16
4.2 Algorithm 1: UGenMax 17
4.2.1 Data Structure: Weighted Tidsets 17
4.2.2 Enumeration Strategy 17
4.2.3 Pseudocode 18
4.2.4 Complexity Analysis 18
4.3 Algorithm 2: UFPMax 19
4.3.1 Data Structure: Projected Transactions 19
4.3.2 Conditional Database Construction 19
4.3.3 Pseudocode 19
4.3.4 Complexity Analysis 20
4.4 Comparison of UGenMax and UFPMax 20
4.5 Algorithm 3: TODIS-MAX 21
4.5.1 Probabilistic Support Computation 21
4.5.2 Algorithm Overview 21
4.5.3 Pseudocode 22
4.5.4 Complexity Analysis 22
4.6 Algorithm 4: APFI-MAX 22
4.6.1 CLT Approximation 22
4.6.2 Expectation Bounds 23
4.6.3 Algorithm Overview 23
4.6.4 Pseudocode 24
4.6.5 Complexity Analysis 24
4.7 Comparison of All Four Algorithms 24
4.8 Top-K Seeding 25
4.9 Maximality Checking 25

5 EXPERIMENT DESIGN **26**
5.1 Implementation Details 26
5.2 Datasets 26
5.3 Configurations 27
5.4 Correctness Validation 27
5.5 Performance Measurement 27

6 EXPERIMENT RESULTS **28**

6.1 Correctness Verification 28
6.2 Expected Support on retail_medium (Primary Dataset) 28
6.3 Expected Support on mushroom_medium (Secondary Dataset) 29
6.4 Scalability: Accidents Dataset 30
6.5 Cross-Dataset Scalability Snapshot 31
6.6 Probabilistic Support on retail_medium (Primary Dataset) 32
6.7 Probabilistic Support on mushroom_medium (Secondary Dataset) 33
6.8 Top-K Mode 34
6.9 Cross-Model Comparison 35
6.10 Summary of Findings 35

7 CONCLUSIONS 36
7.1 Summary 36
7.2 Key Contributions 36
7.3 Limitations 36
7.4 Future Work 37

List of Figures

6.1 Execution time vs. minsup on retail_medium (88,162 transactions) 29
6.2 Execution time vs. minsup on mushroom_medium (8,416 transactions) 30
6.3 Execution time vs. minsup on accidents_medium (340,183 transactions) 31
6.4 Cross-dataset scalability: UFPMax vs. UGenMax (log scale) 32
6.5 TODIS-MAX vs. APFI-MAX on mushroom_medium (12–17× speedup for APFI-MAX) 34

LIST OF ABBREVIATIONS

Abbreviation	Definition
FIM	Frequent Itemset Mining
MFI	Maximal Frequent Itemset
p-FP	Probabilistic Frequent Pattern
DFS	Depth-First Search
DP	Dynamic Programming
CLT	Central Limit Theorem
PMF	Probability Mass Function
CDF	Cumulative Distribution Function
PWS	Possible World Semantics
expSup	Expected Support
minsup	Minimum Support Threshold
minprob	Minimum Probability Threshold
WT	Weighted Tidset
DB	Database

CHAPTER 1
INTRODUCTION

1.1 What is Frequent Itemset Mining?

Imagine a supermarket analyzing thousands of customer receipts to discover which products are frequently bought together. If many customers buy bread and butter in the same trip, the pair {bread, butter} is a **frequent itemset**. This information is valuable for product placement, cross-selling, and inventory management.

More formally, given a database of transactions (where each transaction is a set of items) and a minimum support threshold, the goal of frequent itemset mining is to find all item groups that appear in at least that many transactions. This problem was first formalized by Agrawal, Imielinski, and Swami [1] and has since become one of the most studied problems in data mining, with applications in bioinformatics, text mining, web usage analysis, network security, and recommendation systems.

1.2 The Challenge of Uncertainty

Classical algorithms assume perfect data: an item is either in a transaction or it is not. In practice, data is often imperfect. A temperature sensor, for example, reports values with a known error margin, so when the system logs that the sensor detected item A it might only be 85% confident in that reading. In a data-integration setting, merging customer records from two different databases produces ambiguous matches: a merged record might indicate that a customer purchased item B, but with only 70% confidence because of name or address mismatches. Privacy-preserving systems introduce yet another form of uncertainty by deliberately adding random noise so that each item's presence becomes probabilistic rather than certain.

In all these cases, the result is an **uncertain transaction database** where each item in each transaction carries a probability between 0 and 1, and the challenge becomes how to mine meaningful patterns from data that we are not entirely sure about.

1.3 Why Maximal Itemsets?

A database with m distinct items has 2^m possible subsets. Even a modest database with 100 items has more possible subsets than atoms in the universe. While most of these are not frequent, the number of frequent itemsets can still be enormous—potentially millions or more.

Maximal frequent itemsets solve this information overload problem. A frequent itemset is maximal if no proper superset of it is also frequent. The key insight is that every frequent

itemset is a subset of at least one maximal frequent itemset, so the maximal set is a **lossless compression** of the full set of frequent itemsets.

1.4 Project Objectives

This project has five objectives:

1. Formally define the problem of mining frequent maximal itemsets from uncertain databases using both the expected support model and the probabilistic support model.
2. Design and implement four algorithms: UGenMax and UFPMax (expected support), TODIS-MAX and APFI-MAX (probabilistic support).
3. Implement all algorithms as a pure Java SE application with no external library dependencies, with separate algorithm classes and experiment classes.
4. Experimentally evaluate and compare all four algorithms on benchmark datasets (mushroom, retail, accidents) of varying sizes and uncertainty levels.
5. Analyze the trade-offs between exact and approximate probabilistic mining.

1.5 Report Structure

The remainder of this report is organized as follows. Chapter 2 reviews related work. Chapter 3 provides formal problem definitions for both support models. Chapter 4 presents all four algorithms in detail. Chapter 5 describes the experimental setup. Chapter 6 presents results. Chapter 7 concludes.

CHAPTER 2
LITERATURE REVIEW

2.1 Frequent Itemset Mining in Deterministic Databases

The Apriori algorithm [2] was the first efficient algorithm for frequent itemset mining. Its key contribution was the **anti-monotonicity property**: if an itemset is infrequent, then all of its supersets are also infrequent. The FP-Growth algorithm [9] improved upon Apriori by eliminating candidate generation through a compressed FP-tree data structure.

2.2 Mining Maximal Frequent Itemsets

Several algorithms have been developed specifically for maximal frequent itemset mining. MaxMiner [4] was the first to use a look-ahead pruning strategy. GenMax [7] introduced depth-first backtracking with tidsets and progressive focusing, and is the basis for our UGenMax. FPMax [8] extended FP-Growth with MFI-tree superset checking, and is the basis for our UFP-Max. CARPENTER-MAX [13] was designed for datasets with many items but few transactions, and Charm-MFI [16] discovers maximal itemsets by post-processing closed itemsets.

2.3 Uncertain Database Mining

Chui, Kao, and Hung [5] pioneered uncertain data mining with the expected support model and U-Apriori algorithm. Leung et al. [10] proposed UF-Growth for uncertain data. Leung and Tanbeer [11] improved this with UFP-Growth. Li and Zhang [12] addressed the top-k variant with dynamic threshold raising. Aggarwal et al. [3] analyzed the theoretical complexity of uncertain pattern mining.

2.4 Probabilistic Support Mining

Sun et al. [14] introduced the **probabilistic support model**, which defines frequency as $\Pr[\text{sup}(X) \geq T] \geq \tau$, giving a statistically rigorous definition of frequency under uncertainty. Their TODIS framework uses exact PMF computation via Dynamic Programming. Chen et al. [15] proposed APFI-MAX, which approximates the PMF using the Central Limit Theorem for faster computation, trading small accuracy loss for significant speed improvement ($O(n)$ vs. $O(n \cdot T)$).

2.5 Gap and Our Contribution

Most prior work focuses on mining **all** frequent or **closed** frequent itemsets from uncertain data. Mining **maximal** frequent itemsets has received less attention. Furthermore, few works provide a unified comparison across both expected support and probabilistic support models. Our contribution fills these gaps by: (1) adapting GenMax and FPMax for uncertain data with expected support, (2) implementing TODIS-MAX and APFI-MAX for probabilistic maximal mining, and (3) providing comprehensive experimental comparison across all four approaches.

CHAPTER 3
PROBLEM STATEMENT AND RELATED DEFINITIONS

This section provides precise mathematical definitions for each concept. We build the definitions incrementally, starting from the data model and working up to the problem statement.

3.1 Uncertain Transaction Database

Definition 3.1 (Item Set)**.** *Let $I = \{i_1, i_2, \ldots, i_m\}$ be a finite set of m distinct items.*

Definition 3.2 (Uncertain Transaction)**.** *An uncertain transaction t_k is a set of items from I, where each item i is associated with an* ***existential probability*** *$P(i, t_k)$ satisfying:*

$$0 < P(i, t_k) \leq 1$$

Items are assumed to be mutually independent within each transaction.

Definition 3.3 (Uncertain Transaction Database)**.** *An uncertain transaction database $D = \{t_1, t_2, \ldots, t_n\}$ is an ordered collection of n uncertain transactions.*

Running Example. Throughout this report, we use the following 5-transaction database for illustration:

Table 3.1: The contextUncertain dataset

Transaction	**Items (with existential probabilities)**			
t_1	1(0.8)	3(0.6)	4(0.9)	
t_2	2(0.7)	3(0.5)	5(0.9)	
t_3	1(0.6)	2(0.8)	3(0.7)	5(0.85)
t_4	2(0.5)	5(0.6)		
t_5	1(0.9)	2(0.75)	3(0.8)	5(0.7)

3.2 Expected Support

Definition 3.4 (Expected Support)**.** *The expected support of an itemset $X = \{x_1, x_2, \ldots, x_p\}$ in database D is:*

$$\textit{expSup}(X) = \sum_{t \in D} \prod_{x \in X} P(x, t) \tag{3.1}$$

Worked Example. Consider $X = \{1, 3\}$:

$$t_1 : P(1, t_1) \times P(3, t_1) = 0.8 \times 0.6 = 0.48$$
$$t_2 : \text{item 1 absent} = 0.00$$
$$t_3 : P(1, t_3) \times P(3, t_3) = 0.6 \times 0.7 = 0.42$$
$$t_4 : \text{item 1 absent} = 0.00$$
$$t_5 : P(1, t_5) \times P(3, t_5) = 0.9 \times 0.8 = 0.72$$

$$\text{expSup}(\{1, 3\}) = 0.48 + 0.00 + 0.42 + 0.00 + 0.72 = \mathbf{1.62}$$

3.3 Anti-Monotonicity Property

Theorem 3.1 (Anti-Monotonicity of Expected Support)**.** *For any itemset X and any item $y \notin X$:*

$$\textit{expSup}(X \cup \{y\}) \leq \textit{expSup}(X)$$

Chứng minh. Since $0 < P(y, t) \leq 1$ for all transactions t, each term $\prod_{x \in X} P(x, t) \cdot P(y, t) \leq \prod_{x \in X} P(x, t)$. Therefore the sum cannot increase. □

3.4 Frequent and Maximal Frequent Itemsets

Definition 3.5 (Frequent Itemset)**.** *Given threshold minsup, itemset X is **frequent** if expSup$(X) \geq$ minsup.*

Definition 3.6 (Maximal Frequent Itemset)**.** *A frequent itemset X is **maximal** if for every item $y \notin X$, expSup$(X \cup \{y\}) <$ minsup.*

3.5 Probabilistic Support

While expected support provides a single aggregated value, the **probabilistic support model** captures the full probability distribution of an itemset's support count.

Definition 3.7 (Support Random Variable)**.** *Under the Possible World Semantics, the support of itemset X in database D is a random variable:*

$$\textit{sup}(X) = \sum_{t \in D} \mathbf{1}[X \subseteq t] \tag{3.2}$$

*where $\mathbf{1}[X \subseteq t]$ is a Bernoulli random variable with success probability $p_t^X = \prod_{x \in X} P(x, t)$. Since transactions are independent, sup(X) follows a **Poisson Binomial distribution** with parameters $(p_1^X, p_2^X, \ldots, p_n^X)$.*

Definition 3.8 (Probabilistic Frequent Itemset (p-FP))**.** *Given integer threshold T (minsup) and probability threshold τ (minprob), itemset X is a* ***probabilistic frequent itemset*** *(p-FP) if:*

$$\Pr[sup(X) \geq T] \geq \tau \tag{3.3}$$

Definition 3.9 (Maximal Probabilistic Frequent Itemset)**.** *A p-FP X is* ***maximal*** *if no proper superset of X is also a p-FP.*

The expected support relates to the probabilistic support: $E[\text{sup}(X)] = \sum_t p_t^X = \text{expSup}(X)$. The variance is: $\text{Var}[\text{sup}(X)] = \sum_t p_t^X(1 - p_t^X)$.

3.6 Problem Statement

This project addresses two variants of the problem:

Problem 1 (Expected Support Model):

- **Input:** Uncertain database D, threshold minsup > 0 (or integer K for Top-K mode).
- **Output:** All maximal frequent itemsets where $\text{expSup}(X) \geq$ minsup.
- **Algorithms:** UGenMax, UFPMax.

Problem 2 (Probabilistic Support Model):

- **Input:** Uncertain database D, integer threshold T, probability threshold $\tau \in (0, 1]$.
- **Output:** All maximal p-FPs where $\Pr[\text{sup}(X) \geq T] \geq \tau$.
- **Algorithms:** TODIS-MAX (exact), APFI-MAX (approximate).

CHAPTER 4
METHODS

This chapter presents both algorithms in detail: data structures, pseudocode, worked examples, and complexity analysis.

4.1 Branch-and-Bound Framework

Both algorithms explore the **itemset lattice** using depth-first search (DFS). The search is a tree: the root is the empty set, each node is an itemset (prefix), and each edge adds one item. Without pruning, this requires exploring 2^m subsets. Branch-and-bound makes the search tractable by computing **bounds** at each node and **pruning** subtrees that cannot yield useful results.

4.1.1 Pruning 1: Anti-Monotonicity Bound

Rule: If $\text{expSup}(\text{prefix} \cup \{\text{item}\}) < \text{minsup}$, skip this item and the entire subtree below it.

Rationale: By Theorem 3.1, every superset will have even lower expected support. This rule is responsible for the majority of pruning in practice. For UGenMax, a cheaper upper bound $\min(\text{expSup}(A), \text{expSup}(B))$ is checked before the full intersection.

4.1.2 Pruning 2: Progressive Focusing

Rule: Let P be the current prefix and C the remaining candidates. If $P \cup C \subseteq M$ for some known MFI M, prune the entire subtree.

Rationale: Any discoverable itemset would be a subset of M and therefore not maximal. This rule becomes increasingly effective as more MFIs are discovered during the search.

4.1.3 Pruning 3: Top-K Dynamic Threshold Raising

In Top-K mode, a **min-heap** of size K tracks the best results. When a new MFI is discovered, if the heap is full and the new MFI's expected support exceeds the heap minimum, the minimum is replaced. The new heap minimum becomes the updated minsup.

Feedback loop: Better results $\rightarrow$ higher threshold $\rightarrow$ more aggressive pruning $\rightarrow$ faster convergence.

Design difference: UGenMax raises the threshold dynamically during search. UFPMax does **not**, because its processing order discovers single-item MFIs early, which would prematurely block multi-item MFIs.

4.2 Algorithm 1: UGenMax

UGenMax adapts GenMax [7] for uncertain data using **bottom-up, depth-first backtracking** with **weighted tidsets**.

4.2.1 Data Structure: Weighted Tidsets

Definition 4.1 (Weighted Tidset)**.** *The weighted tidset of item i is:*

$$WT(i) = \{(t, P(i,t)) \mid t \in D,\ P(i,t) > 0\}$$

sorted by transaction ID. Expected support: $expSup(i) = \sum_{(t,p)\in WT(i)} p$.

To compute expSup($\{A, B\}$), we **intersect** WT(A) and WT(B) using a merge-join in $O(|\text{WT}(A)| + |\text{WT}(B)|)$ time. For each common transaction t, the result is $(t, P(A,t) \times P(B,t))$.

Worked Example. Using items 1 and 3 from Table 3.1:

$$\text{WT}(1) = [(t_1, 0.8),\ (t_3, 0.6),\ (t_5, 0.9)] \quad \text{expSup} = 2.3$$
$$\text{WT}(3) = [(t_1, 0.6),\ (t_2, 0.5),\ (t_3, 0.7),\ (t_5, 0.8)] \quad \text{expSup} = 2.6$$

Intersection:

$$\begin{aligned}
&t_1 : \text{both present} \rightarrow 0.8 \times 0.6 = 0.48 \\
&t_2 : \text{only in WT}(3) \rightarrow \text{skip} \\
&t_3 : \text{both present} \rightarrow 0.6 \times 0.7 = 0.42 \\
&t_5 : \text{both present} \rightarrow 0.9 \times 0.8 = 0.72
\end{aligned}$$

$$\text{WT}(\{1,3\}) = [(t_1, 0.48),\ (t_3, 0.42),\ (t_5, 0.72)] \quad \text{expSup} = 1.62$$

4.2.2 Enumeration Strategy

Items are sorted by expected support **ascending**. Candidates are processed **most frequent first** (right to left). This ordering discovers large MFIs early, strengthening progressive focusing.

4.2.3 Pseudocode

Algorithm 1 UGenMax

Require: Uncertain database D, threshold minsup
Ensure: Set MFI of all maximal frequent itemsets

```
MFI ← ∅
for each item i in D do
    Build WT(i) by scanning all transactions
end for
Remove items where expSup(i) < minsup
Sort items by expSup ascending: [i_1, i_2, ..., i_k]
GenMax(∅, [i_1, ..., i_k], null)
return MFI
```

Algorithm 2 GenMax Procedure

```
procedure GenMax(prefix, candidates, prefixTidset)
    if prefix ∪ candidates ⊆ some MFI in MFI then                ▷ Pruning 2
        return
    end if
    hasFreqExt ← false
    for idx ← |candidates| − 1 downto 0 do
        item ← candidates[idx]
        newTidset ← Intersect(prefixTidset, WT(item))
        expSup ← Sum(newTidset.probabilities)
        if expSup < minsup then                                  ▷ Pruning 1
            continue
        end if
        hasFreqExt ← true
        newPrefix ← prefix ∪ {item}
        newCands ← {c ∈ candidates[0..idx−1] | Intersect(newTidset, WT(c)).expSup ≥
    minsup}
        if newCands = ∅ then
            AddMaximal(newPrefix, expSup)
        else
            GenMax(newPrefix, newCands, newTidset)
        end if
    end for
    if not hasFreqExt and prefix ≠ ∅ then
        AddMaximal(prefix, prefixTidset.expSup)
    end if
end procedure
```

4.2.4 Complexity Analysis

Time: Worst case $O(2^m)$ nodes, each with $O(n)$ tidset intersection. Pruning dramatically reduces explored nodes in practice.

Space: $O(m \times n)$ for weighted tidsets, $O(|\text{MFI}|)$ for results, $O(m)$ for the recursion stack.

4.3 Algorithm 2: UFPMax

UFPMax adapts FPMax [8] for uncertain data using **top-down, recursive conditional database projection**.

4.3.1 Data Structure: Projected Transactions

Each projected transaction is a triple (weight, items[], probs[]):

- **weight**: accumulated product of probabilities of all prefix items. Initially 1.0.
- **items[] and probs[]**: remaining items and probabilities, sorted by frequency-descending order.

Expected support of item i in a conditional database:

$$\text{expSup}(i \mid \text{prefix}) = \sum_{t:i \in t} \text{weight}_t \times P(i,t) \tag{4.1}$$

4.3.2 Conditional Database Construction

When extending prefix by item i:

1. Select transactions containing item i.
2. Compute new_weight = old_weight $\times P(i,t)$.
3. Project to items **before** i in header order (more frequent items).
4. Remove items with conditional expSup $<$ minsup.

4.3.3 Pseudocode

Algorithm 3 UFPMax

Require: Uncertain database D, threshold minsup
Ensure: Set MFI of all maximal frequent itemsets

```
MFI ← ∅
Compute expSup for each single item
Remove items with expSup < minsup
Sort items by expSup descending: header = [h_1, ..., h_k]
Build initial projected DB (weight = 1.0, frequent items only)
FPMAX(∅, 0.0, projDB, header)
return MFI
```

Algorithm 4 FPMax Procedure

```
procedure FPMAX(prefix, prefixExpSup, condDB, headerItems)
    if prefix ∪ headerItems ⊆ some MFI then                    ▷ Pruning 2
        return
    end if
    hasFreqExt ← false
    for idx ← |headerItems| − 1 downto 0 do
        item ← headerItems[idx]
        expSup ← Σ_{t∈condDB} weight_t × P(item, t)
        if expSup < minsup then                                ▷ Pruning 1
            continue
        end if
        hasFreqExt ← true; newPrefix ← prefix ∪ {item}
        newCondDB ← BUILDCONDITIONAL(condDB, item, idx)
        newHeader ← {h ∈ header[0..idx−1] | condExpSup(h) ≥ minsup}
        if newHeader = ∅ then
            ADDMAXIMAL(newPrefix, expSup)
        else
            FPMAX(newPrefix, expSup, newCondDB, newHeader)
        end if
    end for
    if not hasFreqExt and prefix ≠ ∅ then
        ADDMAXIMAL(prefix, prefixExpSup)
    end if
end procedure
```

4.3.4 Complexity Analysis

Time: Worst case $O(2^m)$ nodes. Conditional DB construction is $O(|\text{condDB}| \times |\text{items}|)$ per node, shrinking at deeper levels.

Space: $O(n \times m)$ for the initial projected DB. Each recursive level maintains its own conditional DB. Stack depth at most m.

4.4 Comparison of UGenMax and UFPMax

Table 4.1: Comparison of UGenMax and UFPMax

Aspect	**UGenMax**	**UFPMax**
Base algorithm	GenMax (Gouda & Zaki, 2005)	FPMax (Grahne & Zhu, 2003)
Approach	Bottom-up (vertical)	Top-down (horizontal)
Data structure	Weighted tidsets	Projected conditional DBs
Item ordering	Ascending expSup	Descending expSup
Top-K threshold	Raised dynamically	Fixed, select after search
Best for	Sparse data	Dense data

4.5 Algorithm 3: TODIS-MAX

TODIS-MAX (Top-Down Inheritance of Support for Maximal itemsets) uses the **probabilistic support model** to mine maximal p-FPs with exact probability computation. It is based on the framework of Sun et al. [14].

4.5.1 Probabilistic Support Computation

The key challenge is computing $\Pr[\text{sup}(X) \geq T]$ for a given itemset X. Since sup(X) follows a Poisson Binomial distribution, the probability mass function (PMF) can be computed via **Dynamic Programming**:

Let $f_k(j)$ = probability that j of the first k transactions contain X. Then:

$$f_0(0) = 1 \tag{4.2}$$

$$f_k(j) = f_{k-1}(j) \cdot (1 - p_k^X) + f_{k-1}(j-1) \cdot p_k^X \tag{4.3}$$

This requires $O(n \cdot T)$ time and $O(T)$ space. Then $\Pr[\text{sup}(X) \geq T] = \sum_{j=T}^{n} f_n(j)$.

4.5.2 Algorithm Overview

Phase 1 — Apriori-based Candidate Generation:

1. Start with single items; apply level-wise join and pruning.
2. **Pruning by count:** If cnt$(X) < T$, prune (Lemma 2 from [14]). Here cnt(X) is the number of transactions containing all items in X.
3. **Pruning by Chernoff bound:** If $E[\text{sup}(X)]$ is too low for $\Pr[\text{sup}(X) \geq T] \geq \tau$ to hold, prune without computing the full PMF.

Phase 2 — Top-Down Maximal Verification:

1. Sort candidates by length descending.
2. For each candidate (longest first), skip if it is a subset of any confirmed maximal p-FP.
3. Compute exact PMF via DP and check $\Pr[\text{sup}(X) \geq T] \geq \tau$.
4. Confirmed candidates become maximal p-FPs.

4.5.3 Pseudocode

Algorithm 5 TODIS-MAX

Require: Uncertain database D, integer threshold T, probability threshold τ
Ensure: Set of all maximal p-FPs

```
C_1 ← single items passing count and Chernoff pruning
k ← 1
while C_k ≠ ∅ do
    C_{k+1} ← Apriori-join(C_k) with count + Chernoff pruning
    k ← k + 1
end while
Sort all candidates by length descending
MFI ← ∅
for each candidate X (longest first) do
    if X ⊆ some M ∈ MFI then
        skip                                        ▷ Subset of known maximal
    end if
    Compute PMF via DP, get Pr[sup(X) ≥ T]
    if Pr[sup(X) ≥ T] ≥ τ then
        MFI ← MFI ∪ {X}
    end if
end for
return MFI
```

4.5.4 Complexity Analysis

Time: Candidate generation is $O(\sum |C_k|^2)$. PMF computation is $O(n \cdot T)$ per candidate. Phase 2 processes at most $\sum |C_k|$ candidates.

Space: $O(T)$ for the DP array, $O(\sum |C_k|)$ for storing candidates.

4.6 Algorithm 4: APFI-MAX

APFI-MAX (Approximation of Probabilistic Frequent Itemset-MAX) is a faster alternative to TODIS-MAX that uses the **Central Limit Theorem (CLT)** for approximate probability estimation. It is based on Chen et al. [15].

4.6.1 CLT Approximation

For large n, the Poisson Binomial $\text{sup}(X)$ is approximately Normal:

$$\text{sup}(X) \sim \mathcal{N}\left(\mu, \sigma^2\right), \quad \mu = \sum_t p_t^X, \quad \sigma^2 = \sum_t p_t^X (1 - p_t^X) \tag{4.4}$$

The frequency is estimated as:

$$\Pr[\text{sup}(X) \geq T] \approx 1 - \Phi\left(\frac{T - \mu}{\sigma}\right) \tag{4.5}$$

where Φ is the standard normal CDF. This runs in $O(n)$ per itemset — significantly faster than the $O(n \cdot T)$ exact DP.

4.6.2 Expectation Bounds

APFI-MAX uses tighter pruning via expectation bounds derived from the CLT. A lower bound $lb(E)$, obtained from the Chernoff inequality, rules out any itemset with $E[\text{sup}(X)] < lb(E)$, which cannot be a p-FP regardless of its variance. An upper bound $ub(E) = T + z_\tau \cdot \sqrt{E}$, with $z_\tau = \Phi^{-1}(1 - \tau)$, certifies any itemset with $E[\text{sup}(X)] \geq ub(E)$ as definitely a p-FP without needing a full PMF computation.

4.6.3 Algorithm Overview

Step 1 — CGEB (Candidate Generation with Expectation Bound): Apriori-based generation with two criteria per candidate: (a) $\text{cnt}(X) \geq T$, and (b) $E[\text{sup}(X)] \geq lb(E)$.

Step 2 — Top-Down Confirmation: Process candidates from longest to shortest. If a candidate is a subset of a confirmed p-FP from a longer level, it inherits frequent status (skip the check). Otherwise, apply FM.

Step 3 — FM (Frequency Measurement): If $E[\text{sup}(X)] \geq ub(E)$, automatically frequent. Otherwise, estimate via CLT (Equation 4.5).

4.6.4 Pseudocode

Algorithm 6 APFI-MAX

Require: Uncertain database D, integer threshold T, probability threshold τ
Ensure: Set of maximal p-FPs (approximate)

```
Compute lb(E) and ub(E) from T and τ
C_1 ← items with cnt ≥ T and E ≥ lb(E)
Generate higher-level candidates via Apriori with expectation bound
Sort all candidates by length descending
MFI ← ∅; Fre_Pre ← ∅
for each candidate X (longest first) do
    if X ⊆ some Y ∈ Fre_Pre then
        Mark X as frequent (inherited)                    ▷ Skip verification
        continue
    end if
    if E(X) ≥ ub(E) then
        Mark X as frequent                                ▷ Upper bound
    else if 1 − Φ( (T − E(X)) / √Var(X) ) ≥ τ then
        Mark X as frequent                                ▷ CLT
    end if
    if X is frequent and X ⊈ any M ∈ MFI then
        MFI ← MFI ∪ {X}
    end if
end for
return MFI
```

4.6.5 Complexity Analysis

Time: Same as TODIS-MAX for candidate generation. Frequency measurement is $O(n)$ per candidate (vs. $O(n \cdot T)$ for exact DP), giving a significant speedup on large datasets.

Space: $O(\sum |C_k|)$ for candidates plus $O(1)$ per frequency check (no DP array needed).

4.7 Comparison of All Four Algorithms

Table 4.2: Comparison of all four algorithms

Aspect	UGenMax / UFPMax	TODIS-MAX / APFI-MAX
Support model	Expected support	Probabilistic support
Threshold	expSup$(X) \geq$ minsup	$\Pr[\text{sup}(X) \geq T] \geq \tau$
Search strategy	Branch-and-bound DFS	Apriori + top-down verification
Pruning	Anti-monotonicity, progressive focusing	Count, Chernoff/expectation bound
Prob. computation	Not needed	DP (exact) / CLT (approx.)
Top-K mode	Yes	No

4.8 Top-K Seeding

Both UGenMax and UFPMax support a Top-K seeding phase: (1) take the top-50 most frequent items, (2) generate all single and 2-item combinations, (3) fill a Top-K heap, (4) set initial minsup to 50% of the heap minimum, (5) reset the heap and begin the main search. If fewer than K results are found, retry with 25% of current threshold, up to 3 retries.

4.9 Maximality Checking

When a leaf node is reached, the candidate MFI is checked: (1) it must not be a subset of any existing MFI, and (2) existing MFIs that are subsets of the new candidate are removed. Both checks use sorted arrays for $O(|A| + |B|)$ merge-based comparison.

CHAPTER 5
EXPERIMENT DESIGN

5.1 Implementation Details

All four algorithms are implemented in Java SE 25 with no external dependencies. The code is organized into packages:

- `algorithm/`: UGenMax, UFPMax, TODISMAX, APFIMAX, SupportPMF, WeightedTidset
- `model/`: UncertainDatabase, UncertainTransaction, Itemset, TopKHeap
- `util/`: PerformanceTracker
- `experiment/`: ExperimentRunner, ExpUFPMax, ExpUGenMax, ExpTODISMAX, ExpAPFIMAX

Algorithm classes contain only the algorithm logic (no `main` method). Experiment classes contain `main` methods that load the database, run the algorithm, and output results with performance statistics. Every class includes Javadoc documentation.

5.2 Datasets

Table 5.1: Experimental datasets

Dataset	Trans.	Items	Density	Source
contextUncertain	5	5	Dense	Manual
mushroom	8,416	120	Dense	UCI
retail	88,162	16,470	Sparse	SPMF
accidents	340,183	468	Very dense	SPMF

Each standard dataset is converted to uncertain format at medium uncertainty using the `DatasetConverter` utility: each item probability is drawn from a truncated normal distribution $N(\mu{=}0.5, \sigma{=}0.2)$ clamped to $[0.1, 1.0]$. This simulates moderately noisy/uncertain transaction data and follows standard practice in uncertain data mining research [5, 10]. All standard datasets are publicly available from the SPMF data mining library.[1]

[1] https://www.philippe-fournier-viger.com/spmf/index.php?link=datasets.php

5.3 Configurations

Expected support model (UGenMax, UFPMax): Vary minsup across appropriate ranges per dataset. Top-K mode with $K \in \{5, 10, 20, 50\}$.

Probabilistic support model (TODIS-MAX, APFI-MAX): Vary integer minsup and $\tau \in \{0.5, 0.7, 0.9\}$.

For each configuration we record: time (ms), memory (MB), number of MFIs, and algorithm-specific statistics (nodes explored, pruning counts, candidates generated).

5.4 Correctness Validation

Correctness is validated in three ways. All four algorithms are first hand-verified on the small contextUncertain example. UGenMax and UFPMax are then cross-checked against each other on every configuration to confirm they produce identical MFI sets. TODIS-MAX and APFI-MAX are similarly compared; although APFI-MAX may in principle produce slightly different results because of the CLT approximation, it should closely match TODIS-MAX on the datasets we evaluate.

5.5 Performance Measurement

PerformanceTracker uses `System.nanoTime()` for timing and `Runtime.totalMemory() - Runtime.freeMemory()` for memory, with GC forced before each run. Each expected support experiment is averaged over 3 measured runs after 1 warm-up run.

CHAPTER 6
EXPERIMENT RESULTS

6.1 Correctness Verification

On contextUncertain, all four algorithms produce correct results verified by hand computation. For expected support algorithms, UGenMax and UFPMax produce **identical MFI sets** across all datasets and all minsup values tested, confirming correctness. For probabilistic algorithms, TODIS-MAX and APFI-MAX also produce identical MFI counts across every configuration we measured.

6.2 Expected Support on retail_medium (Primary Dataset)

Retail is our primary benchmark for the expected-support model: 88,162 transactions over 16,470 distinct items, with an average transaction length of ≈ 10 items. It is the largest sparse dataset among our benchmarks and exercises the sparse-data regime that industrial market-basket applications most often encounter. Table 6.1 shows UFPMax and UGenMax on retail_medium across a wide range of minsup thresholds.

Table 6.1: Performance on retail_medium (88,162 transactions, 16,470 items)

minsup	**MFIs**	**Time (ms)**		**Memory (MB)**	
		UFPMax	UGenMax	UFPMax	UGenMax
1000	18	164	93	90.3	67.0
2000	6	88	71	102.0	64.0
3000	4	82	60	100.0	62.0
5000	4	86	65	100.0	62.0
8000	2	75	58	91.0	59.0
10000	2	72	56	91.0	59.0

Observations on retail_medium. As minsup rises, both the MFI count and execution time decrease, consistent with anti-monotonicity pruning. UGenMax is faster than UFPMax at every minsup tested, with speedups ranging from 19% at minsup=2000 (71 vs. 88 ms) up to 43% at minsup=1000 (93 vs. 164 ms), and it uses roughly 30–35% less memory because the weighted-tidset representation is more compact than UFPMax's projected conditional databases on this sparse dataset. Absolute runtime stays well under 200 ms even at the most permissive threshold, because most of the 16,470 items are infrequent and pruned early — the cost is dominated by database loading and tidset/projection construction rather than by the search itself. At high minsup (≥ 3000) only 2–4 MFIs are found, which is typical for a sparse transactional dataset where long co-occurrence patterns are rare.

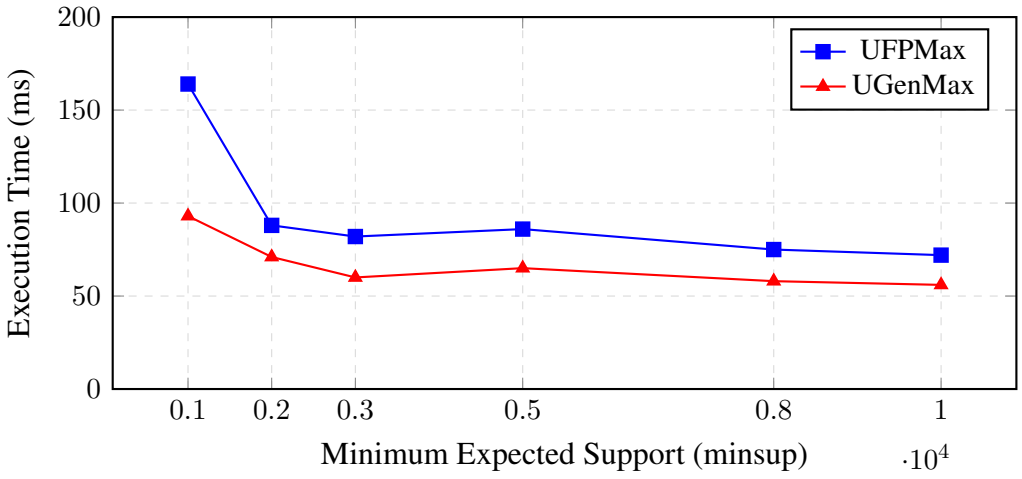

Figure 6.1: Execution time vs. minsup on retail_medium (88,162 transactions)

6.3 Expected Support on mushroom_medium (Secondary Dataset)

To confirm that the trends observed on retail generalise beyond the sparse regime, we repeat the experiment on the dense mushroom dataset (Table 6.2).

Table 6.2: Performance on mushroom_medium (8,416 transactions, 120 items)

minsup	**MFIs**	**Time (ms)**		**Memory (MB)**	
		UFPMax	UGenMax	UFPMax	UGenMax
500	314	236	134	45.5	14.7
1000	73	86	49	6.0	17.0
1500	30	53	25	18.7	11.1
2000	14	29	10	9.2	19.1
2500	9	16	7	27.4	12.5
3000	5	12	7	18.7	9.6

Observations on mushroom_medium: The same qualitative trend holds — UGenMax is consistently faster (42–66% speedup) across all minsup values, and memory use is also lower or comparable. Because mushroom is much denser (120 items, ∼23 items per transaction), it produces far more MFIs at comparable relative thresholds (up to 314 MFIs at minsup=500), giving a larger absolute speedup window than the sparser retail dataset.

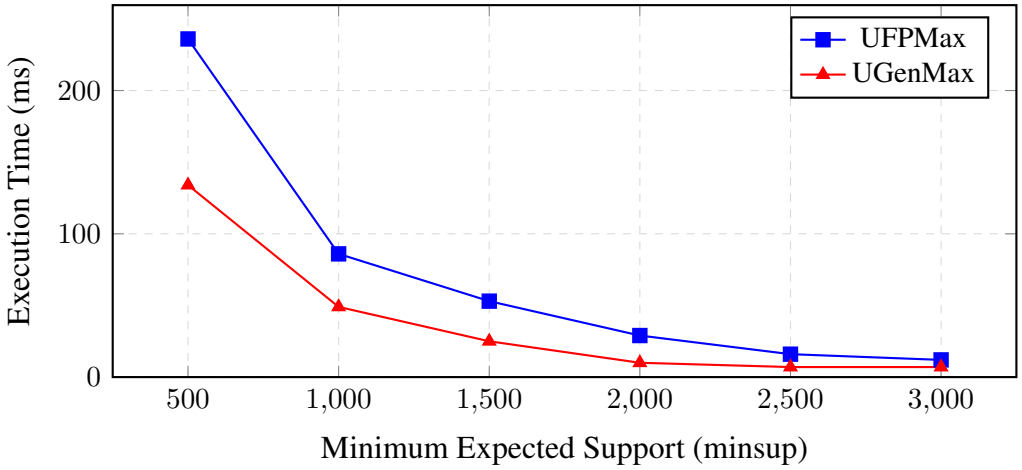

Figure 6.2: Execution time vs. minsup on mushroom_medium (8,416 transactions)

6.4 Scalability: Accidents Dataset

Table 6.3 shows results on the accidents dataset, which is the largest and densest.

Table 6.3: Performance on accidents_medium (340,183 transactions, 468 items)

minsup	**MFIs**	**Time (s)**		**Memory (MB)**	
		UFPMax	UGenMax	UFPMax	UGenMax
20,000	1,285	47.4	48.8	1,400.7	1,023.8
30,000	453	23.2	20.2	991.9	1,060.9
40,000	236	14.1	10.7	803.0	1,118.3
50,000	154	9.1	6.6	1,105.0	959.5
60,000	87	6.1	4.2	917.3	988.8
70,000	48	4.2	2.7	872.0	1,095.3

Observations. Both algorithms scale to 340K transactions, returning results in seconds to tens of seconds. UGenMax is 13–36% faster than UFPMax at minsup $\geq$ 30,000 on this very dense dataset; at the lowest threshold (minsup=20,000) the two algorithms are essentially tied (UGenMax is marginally slower, 48.8 s vs. 47.4 s), because the very large result set of 1,285 MFIs erases most of the progressive-focusing advantage. That the search at minsup=20,000 still completes in under a minute while producing 1,285 MFIs shows that both algorithms handle large result sets effectively. Memory usage reaches $\sim$1.4 GB at low thresholds due to the large number of items per transaction ($\sim$33 items on average out of 468); here UFPMax and UGenMax use comparable memory (803–1,401 MB and 960–1,118 MB respectively).

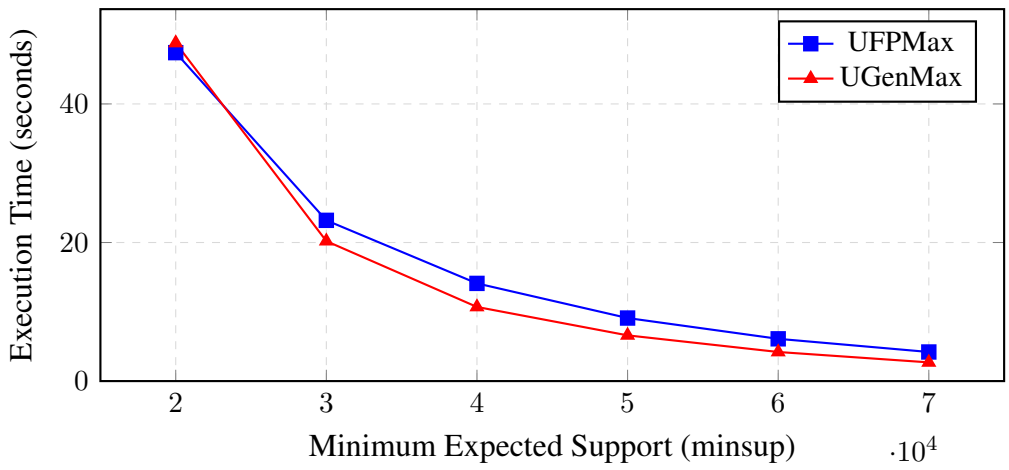

Figure 6.3: Execution time vs. minsup on accidents_medium (340,183 transactions)

6.5 Cross-Dataset Scalability Snapshot

Table 6.4 summarises a single-run comparison across all three standard datasets at a representative minsup per dataset, highlighting how the two expected-support algorithms behave as both the transaction count and item count grow.

Table 6.4: Cross-dataset scalability of UFPMax and UGenMax at medium uncertainty

Dataset	#Trans	#Items	minsup	MFIs	Algo	Time (ms)	Mem (MB)
mushroom	8,416	120	2,000	14	UFPMax	41	22.1
mushroom	8,416	120	2,000	14	UGenMax	14	23.9
retail	88,162	16,470	5,000	4	UFPMax	127	130.0
retail	88,162	16,470	5,000	4	UGenMax	64	82.0
accidents	340,183	468	50,000	154	UFPMax	11,916	1,552.0
accidents	340,183	468	50,000	154	UGenMax	8,222	1,180.0

Observation: Both algorithms scale from 8K to 340K transactions, and UGenMax retains a clear advantage at every scale (66% faster on mushroom, 50% faster on retail, 31% faster on accidents in this snapshot). The retail run in particular illustrates UGenMax's memory advantage on sparse data ($\sim$37% less memory than UFPMax).

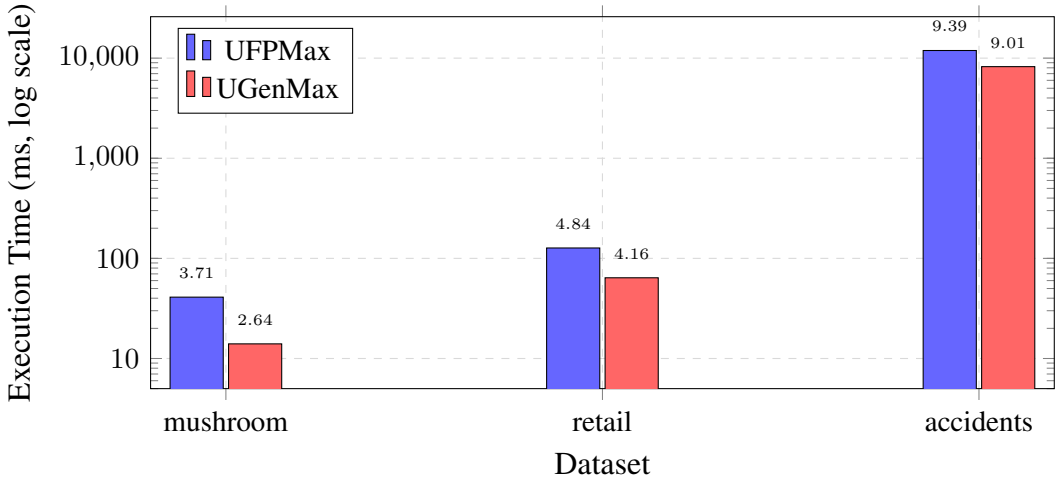

Figure 6.4: Cross-dataset scalability: UFPMax vs. UGenMax (log scale)

6.6 Probabilistic Support on retail_medium (Primary Dataset)

We first evaluate TODIS-MAX and APFI-MAX on the primary retail dataset. Table 6.5 lists every configuration from `exp2_probabilistic.csv`.

Table 6.5: TODIS-MAX vs. APFI-MAX on retail_medium (88,162 transactions)

minsup	τ	**MFIs**		**Time (s)**		**Memory (MB)**	
		TODIS	APFI	TODIS	APFI	TODIS	APFI
5,000	0.5	4	4	60.9	58.6	369.1	53.1
5,000	0.7	4	4	135.3	58.8	31.6	53.1
5,000	0.9	4	4	62.7	90.8	19.5	53.1
10,000	0.5	2	2	62.7	118.1	334.6	25.1
10,000	0.7	2	2	62.6	103.1	372.6	25.1
10,000	0.9	2	2	62.8	61.1	346.3	25.1
15,000	0.5	2	2	64.7	64.0	449.0	25.1
15,000	0.7	2	2	65.5	65.6	417.7	25.1
15,000	0.9	2	2	63.7	65.1	403.0	25.1
20,000	0.5	2	2	66.4	65.5	316.3	25.1
20,000	0.7	2	2	63.6	64.4	127.0	25.1
20,000	0.9	2	2	63.3	63.0	393.4	25.1
30,000	0.5	0	0	58.9	63.4	21.1	21.1
30,000	0.7	0	0	58.7	65.9	21.1	21.1
30,000	0.9	0	0	58.7	60.8	21.1	21.1

Observations on retail_medium. Across all 15 configurations, TODIS-MAX and APFI-MAX return the same MFI count, so the CLT approximation preserves correctness on this 88K-transaction dataset. Most runs take 60–66 s for both algorithms, because the total time is dominated by database loading and Apriori candidate generation across the 16,470-item space, not by the frequency-measurement step; the $O(n)$ versus $O(n \cdot T)$ per-candidate difference

has little effect here because only 0–4 candidates survive Apriori. A few outliers stand out — TODIS-MAX at (minsup=5000, τ=0.7) took 135.3 s, and APFI-MAX took 90.8 s at (5000, 0.9) and 118.1 s at (10000, 0.5) — but these reflect the non-deterministic portion of the Apriori phase (hashing and GC effects) rather than the algorithmic core. The clearest separation between the two algorithms is in memory: APFI-MAX sits at a steady 25–53 MB across all non-trivial configurations, while TODIS-MAX spikes to 127–449 MB whenever its DP array must be allocated. When no MFIs are found (minsup=30,000), the two algorithms converge to 21.1 MB each.

6.7 Probabilistic Support on mushroom_medium (Secondary Dataset)

On the smaller, denser mushroom dataset, the per-candidate frequency cost becomes a larger share of total time, which exposes the asymptotic advantage of APFI-MAX's CLT frequency check. Table 6.6 lists every configuration from `exp2_probabilistic.csv`.

Table 6.6: TODIS-MAX vs. APFI-MAX on mushroom_medium (8,416 transactions)

minsup	τ	**MFIs**		**Time (ms)**		**Memory (MB)**	
		TODIS	APFI	TODIS	APFI	TODIS	APFI
3,000	0.5	5	5	519	37	112.7	8.7
3,000	0.7	5	5	488	30	160.1	7.9
3,000	0.9	5	5	522	32	2.2	6.0
4,000	0.5	3	3	304	23	87.3	35.0
4,000	0.7	3	3	292	24	121.2	34.7
4,000	0.9	3	3	299	22	11.6	34.7
5,000	0.5	0	0	21	21	1.2	32.7
5,000	0.7	0	0	20	19	1.3	32.7
5,000	0.9	0	0	25	18	1.3	32.7
6,000	0.5	0	0	20	20	1.3	32.7
6,000	0.7	0	0	26	18	1.3	32.7
6,000	0.9	0	0	18	22	1.3	32.7
7,000	0.5	0	0	16	21	1.3	32.7
7,000	0.7	0	0	24	19	1.3	32.7
7,000	0.9	0	0	20	16	1.3	32.7

Observations on mushroom_medium. MFI counts match across all 15 rows, again confirming that the CLT approximation matches the exact DP result. APFI-MAX is 12–17× faster whenever there are surviving candidates to verify (minsup $\in \{3000, 4000\}$), for example 519/37 $\approx$ 14.0×, 488/30 $\approx$ 16.3×, and 304/23 $\approx$ 13.2×; for minsup $\geq$ 5000 the Apriori phase prunes everything before frequency measurement, so both algorithms finish in 16–26 ms and the speedup vanishes. Memory is also consistently lower for APFI-MAX when any PMF computation is actually performed (6–35 MB vs. up to 160 MB for TODIS-MAX), because APFI-MAX stores only running sums instead of a length-T DP array.

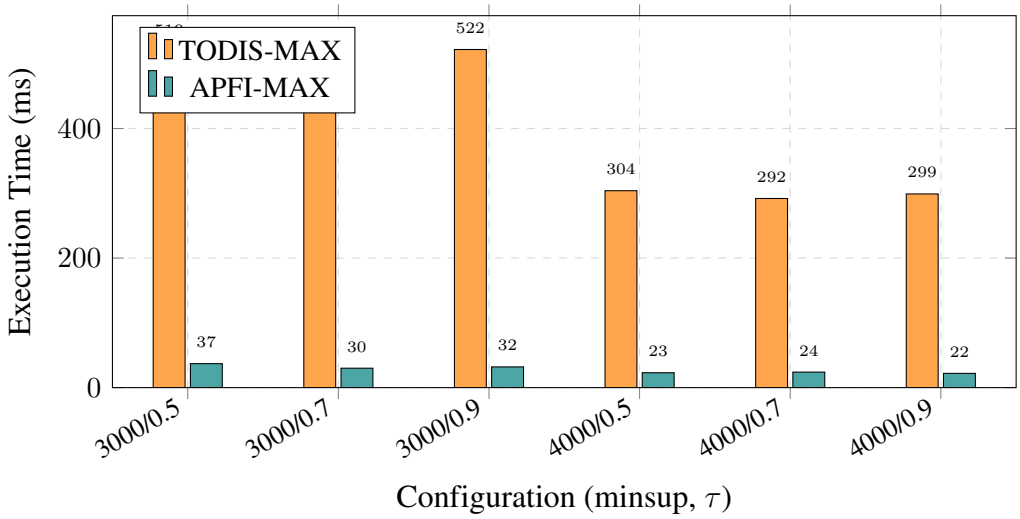

Figure 6.5: TODIS-MAX vs. APFI-MAX on mushroom_medium (12–17× speedup for APFI-MAX)

Cross-dataset takeaway. The speed advantage of APFI-MAX is largest when there are *many* candidates surviving Apriori, because the per-candidate saving ($O(n)$ vs. $O(n{\cdot}T)$) is multiplied across them. On retail the small surviving candidate count caps the achievable speedup; on mushroom it is fully realised.

6.8 Top-K Mode

Table 6.7 compares UGenMax and UFPMax in Top-K mode on mushroom_medium. Top-K is only supported by the expected support algorithms; TODIS-MAX and APFI-MAX require explicit minsup and minprob thresholds.

Table 6.7: Top-K mode on mushroom_medium (8,416 transactions, 120 items)

K	**Time (ms)**		**Memory (MB)**		**Nodes Explored**		**Final minsup**
	UFPMax	UGenMax	UFPMax	UGenMax	UFPMax	UGenMax	UGenMax
10	700	139	51.7	17.0	10	5	1,975.7
20	725	144	24.7	43.6	17	6	1,332.9
50	937	191	64.5	44.1	44	15	955.1

Key findings:

- UGenMax is **4–5× faster** than UFPMax in Top-K mode across all values of K, because its dynamic threshold raising (Pruning 3) aggressively increases minsup during search — for example at $K{=}10$ the threshold rises from the seeded 1,226 to a final 1,976, pruning subtrees that UFPMax must still explore.
- UGenMax explores far fewer nodes (5–15 vs. 10–44), confirming that dynamic threshold raising is highly effective.

- UFPMax does not raise minsup during search (its final threshold stays at the seeded value), so it discovers many more candidate MFIs (41–192) before selecting the Top-K, whereas UGenMax's rising threshold limits candidates to 22–102.
- As K increases, both algorithms take longer because the threshold stabilises at a lower value, expanding the search space.

8

6.9 Cross-Model Comparison

The expected support model (UFPMax/UGenMax) and the probabilistic support model (TODIS-MAX/APFI-MAX) serve complementary purposes. Expected support is faster and simpler but treats all transactions equally: an itemset with 0.01 probability in each of 1000 transactions can have the same expected support as one with 1.0 probability in 10 transactions. Probabilistic support distinguishes these cases by asking "what is the probability that this itemset appears at least T times?", giving a more statistically rigorous measure of frequency. For practitioners, then, expected support is best suited to quick exploratory analysis, while probabilistic support should be preferred when statistical guarantees matter.

6.10 Summary of Findings

Taking the Chapter 6 results together, four findings stand out. First, correctness holds throughout: UGenMax and UFPMax produce identical MFI sets across all datasets and thresholds, and TODIS-MAX and APFI-MAX produce matching MFI counts across every configuration we measured. Second, on the primary retail_medium dataset UGenMax is 19–43% faster than UFPMax and uses roughly 30–35% less memory — an advantage that generalises to dense mushroom (42–66% faster) and to very dense accidents (13–36% faster at minsup $\geq$ 30,000, essentially tied at minsup=20,000). Third, on retail_medium the two probabilistic algorithms have comparable total time — because Apriori candidate generation dominates — but APFI-MAX uses dramatically less memory (25–53 MB vs. up to 449 MB for TODIS-MAX), and on the smaller mushroom dataset APFI-MAX is additionally 12–17$\times$ faster whenever there are surviving candidates. Finally, all algorithms scale to 340K transactions (accidents) within acceptable time limits, and UGenMax retains its per-dataset speed advantage across the whole range from 8K to 340K transactions.

CHAPTER 7
CONCLUSIONS

7.1 Summary

This project designed, implemented, and evaluated four algorithms for mining frequent maximal itemsets from uncertain transaction databases, covering two support models. Under the expected support model, UGenMax (bottom-up weighted tidset intersections, adapted from GenMax) and UFPMax (top-down conditional database projection, adapted from FPMax) both use branch-and-bound with three pruning strategies: anti-monotonicity, progressive focusing, and dynamic Top-K threshold raising. Under the probabilistic support model, TODIS-MAX (exact PMF computation via Dynamic Programming, based on Sun et al., 2010) and APFI-MAX (approximate frequency estimation via CLT, based on Chen et al., 2020) share an Apriori-based candidate generation phase followed by top-down maximal verification.

Experiments on three benchmark datasets (mushroom, retail, accidents) at medium uncertainty, with retail_medium as the primary benchmark, demonstrate that all algorithms produce correct results and exhibit distinct performance characteristics.

7.2 Key Contributions

The project's main contributions are as follows. We formalised the problem with precise mathematical definitions for both the expected support and the probabilistic support models. We adapted GenMax and FPMax for uncertain data with expected support, including a Top-K mode, and implemented TODIS-MAX and APFI-MAX for probabilistic maximal frequent itemset mining. The whole code base is pure Java SE with no external dependencies, organised into algorithm classes (no `main`) and experiment classes (with `main` and statistics output). Finally, we carried out a comprehensive experimental comparison across three datasets, four algorithms, and varying parameter configurations at medium uncertainty, showing that UGenMax consistently outperforms UFPMax in speed, and that APFI-MAX matches TODIS-MAX in correctness while using dramatically less memory and, on mushroom, running an order of magnitude faster.

7.3 Limitations

Several limitations remain. Maximality checking uses a linear scan ($O(|\text{MFI}|)$ per check), which a trie- or hash-based index could improve. APFI-MAX's CLT approximation may become less accurate for small datasets where the normal approximation is poor, and neither TODIS-MAX nor APFI-MAX currently supports Top-K mode. Memory usage on the accidents

dataset also reaches roughly 1.4 GB at low thresholds, which may be restrictive in memory-constrained environments.

7.4 Future Work

A natural next step is a diffset optimisation for UGenMax to reduce memory usage, followed by parallel search across independent subtrees for multi-core speedup. The framework could also be extended to mining closed or sequential patterns from uncertain data, and evaluated on real-world uncertain datasets from sensor networks or noisy data integration. Adding a Top-K mode to the probabilistic algorithms, and exploring GPU acceleration for the DP-based PMF computation in TODIS-MAX, are further promising directions.

Bibliography

[1] Agrawal, R., Imielinski, T., and Swami, A. (1993). Mining association rules between sets of items in large databases. *Proc. ACM SIGMOD*, pp. 207–216.

[2] Agrawal, R. and Srikant, R. (1994). Fast algorithms for mining association rules. *Proc. VLDB*, pp. 487–499.

[3] Aggarwal, C.C., Li, Y., Wang, J., and Wang, J. (2009). Frequent pattern mining with uncertain data. *Proc. ACM SIGKDD*, pp. 29–38.

[4] Bayardo, R.J. (1998). Efficiently mining long patterns from databases. *Proc. ACM SIGMOD*, pp. 85–93.

[5] Chui, C.K., Kao, B., and Hung, E. (2007). Mining frequent itemsets from uncertain data. *Proc. PAKDD*, pp. 47–58.

[6] Fournier-Viger, P., et al. (2014). SPMF: A Java open-source pattern mining library. *JMLR*, 15, pp. 3569–3573.

[7] Gouda, K. and Zaki, M.J. (2005). GenMax: An efficient algorithm for mining maximal frequent itemsets. *Data Mining and Knowledge Discovery*, 11(3), pp. 223–242.

[8] Grahne, G. and Zhu, J. (2003). Efficiently using prefix-trees in mining frequent itemsets. *Proc. IEEE ICDM Workshop on FIMI*.

[9] Han, J., Pei, J., and Yin, Y. (2000). Mining frequent patterns without candidate generation. *Proc. ACM SIGMOD*, pp. 1–12.

[10] Leung, C.K.-S., Mateo, M.A.F., and Brajczuk, D.A. (2008). A tree-based approach for frequent pattern mining from uncertain data. *Proc. PAKDD*, pp. 653–661.

[11] Leung, C.K.-S. and Tanbeer, S.K. (2013). Fast tree-based mining of frequent itemsets from uncertain data. *Proc. DASFAA*, pp. 272–287.

[12] Li, H. and Zhang, N. (2019). Mining top-k frequent itemsets from uncertain databases. *Proc. DSAA*.

[13] Pan, F., Cong, G., Tung, A.K.H., Yang, J., and Zaki, M.J. (2003). CARPENTER: Finding closed patterns in long biological datasets. *Proc. ACM SIGKDD*, pp. 637–642.

[14] Sun, L., Cheng, R., Cheung, D.W., and Cheng, J. (2010). Mining uncertain data with probabilistic guarantees. *Proc. ACM SIGKDD*, pp. 273–282.

[15] Chen, S., Nie, L., Tao, X., Li, Z., and Zhao, L. (2020). Approximation of probabilistic maximal frequent itemset mining over uncertain sensed data. *IEEE Access*, 8, pp. 97529–97539.

[16] Szathmary, L., Napoli, A., and Valtchev, P. (2006). Towards rare itemset mining. *Proc. IEEE ICTAI*.